

BioInfoAggregator.jl: A toolkit for relational bioinformatics data

Nick Sexson¹, Yifeng Yuan¹, Melinda Donahey¹, Valerie De Crecy¹, and Raquel Dias¹

¹ Department of Microbiology and Cell Science, University of Florida, Gainesville, Florida, USA

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [✉](#)

Submitted: 31 July 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

BioInfoAggregator.jl is a Julia language toolkit comprised of utilities for downloading and transforming data from common bioinformatics sources and writing that data to a local relational database. The package also provides an example schema for supporting analyses focused on protein data. While this is given as an example, this package contains modular building blocks to support incorporation and analysis of many types of data. BioInfoAggregator.jl is intended to make it easy to utilize common bioinformatics APIs such as those from uniprot, Alphafold protein structure database, or others to fetch data in JSON format, transform the data, and write the data to a relational database. The primary advantage is that data from multiple sources can be written to the same database and efficiently queried based on user defined relationships. By facilitating a user directed local database, users can even include their own data such as experimental results, custom generated embeddings, or output result from other analytic processes. This allows for complex analysis such as determining the most frequently observed GO terms for protein groups clustered using sequence embeddings by interpro family with a single SQL query.

The default database used by BioInfoAggregator.jl is DuckDB via the DuckDB.jl package. The Julia language is particularly well suited to this use case due to the existence of DBInterface.jl which allows any package that uses this interface to serve as the conduit between the Julia process and the database and thus gives user many options to use for underlying database engine. In addition to DuckDB there are DBInterface.jl compliant packages for SQLite, PostgreSQL, MySQL, and MongoDB. DuckDB was chosen as the default due to its fast vectorized query engine and columnar storage format.

Statement of Need

Modern bioinformatic analyses often draw on data from many different sources. The proliferation of freely available APIs have made data easy to retrieve but have also brought with them the proliferation of so called “file zoos” in which projects often have many directories holding many different files and filetypes which were retrieved ad hoc from various sources all for different parts of the analytic workflow. This requires users to be diligent in folder organization and demands that scripts use relative path references if portability is needed. Databases have long been recognized as a functional solution by ingesting the majority of data directly into the database, and where filetypes demand, give users a clean interface for tracking file locations and metadata within the database even if the files are stored in a separate location. In fact, all of the API’s that serve data to users store the data within databases and use queries for retrieval. The reason that custom databases are not often used in individual projects or analyses is because of the added effort to set up the database in the first place.

BioInfoAggregator.jl aims to simplify that process by providing data downloading utility

functions for common bioinformatics sources, along with schema definitions and ingestion functions to streamline the process of fetching data from sources and adding to a local database. The use of DuckDB(Raasveldt & M"uhleisen, 2019) as the default database minimizes the overhead of setting up a database as it is an in process database, not requiring a server, that also stores all data in a single file. BioInfoAggregator.jl is extensible and allows users to define new download and transformation functions and since the backing storage engine is a database, new tables and relationships can easily be created to incorporate new data. By shifting the responsibility for data organization from the user to the backing database, users can spend more time on the analytical process and less time managing directories, file names, and other "housekeeping" tasks related to data management. Analyses can be run as SQL queries which, in conjunction with the database engine, make analyses shareable and reproducible. The data and analyses can be shared in two files: the database file and an SQL query file.

Some similar tools exist such as BioStructures.jl(Greener & Sternberg, 2021) which is focused on downloading and working with PDB files. BioServices.jl(Cokelaer et al., 2013) is a package for interfacing with the NCBI Entrez databases, NIH UMLS, and GGGEnome database. It is focused on being an interface to these specific APIs and leaves storage and organization of retrieved data to the user. BioFetch.jl(Contributors, 2023) is an annotation based sequence retrieval tool for NCBI Entrez database, UniProt, and Ensembl which retruns data in either FASTA or genebank format.

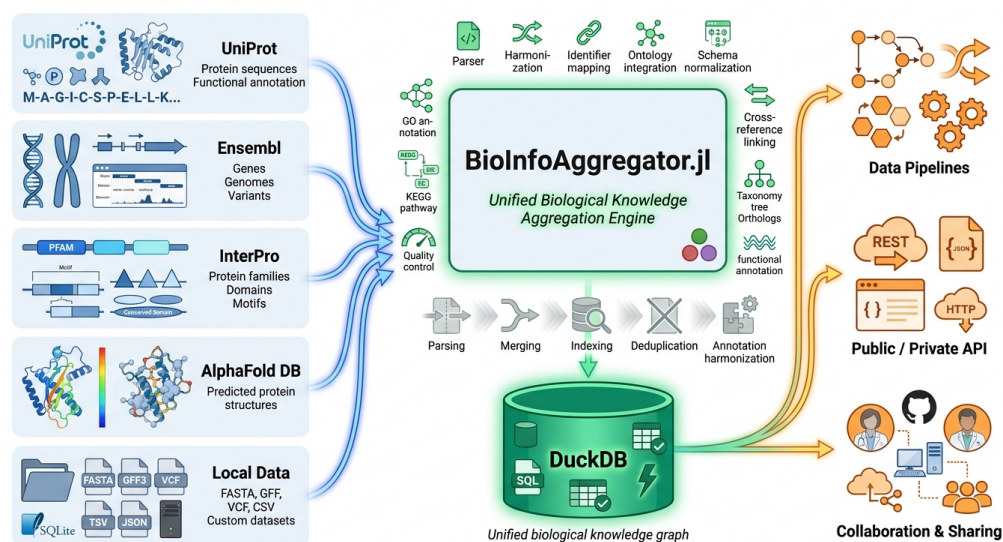


Figure 1: Flowchart

Advantages

BioInfoAggregator.jl is generalizable, multi-purpose, and extensible. A new source can be added by defining a new DBTableSpec object. The table spec struct defines the table schema, name, and how data from the source should be transformed and inserted into the table. The downloader framework is also extensible. Downloader structs specify how to fetch data from a remote source. Single, batch, and paginated downloaders are available by default, but the abstract Downloader type can be extended to add functionality such as authorization, rate limiting, or retries. BioInfoAggregator.jl comes with built in support for fetching data from UniProt(The UniProt Consortium, 2023), Ensembl(Martin et al., 2024), and InterPro[interpro2023].

Beyond extensibility, BioInfoAggregator.jl implements a local storage database that can be used to eliminate the need to make additional API calls. The local database can also be used

by other systems and processes, and is itself extensible, allowing users to add additional tables as needed, within the same database where downloaded data is stored. Using DuckDB as the database also makes the entire database a single file for portability.

Example Use

To download metadata for all members of an interpro family:

```
interproData = fetchDownloads(interproFamilyMemberDownloader,["IPR000098"])
```

```
julia>
```

```
1061×14 DataFrame
```

Row	metadata_accession	metadata_name	metadata_sou ...
	String	String	String
1	A0A023VU42	Viral interleukin-10 homolog	unreviewed
2	A0A059WM99	Interleukin family protein	unreviewed
3	A0A061HZR3	Interleukin family protein	unreviewed
4	A0A068EG07	Viral interleukin-10 homolog	unreviewed
5	A0A068EGM0	Viral interleukin-10 homolog	unreviewed

Reading data from the local DB:

```
readDB(interproFamilyMemberDownloader)
```

```
>julia
```

```
1061×12 DataFrame
```

Row	uniprot_accession	name	source_database	length	source
	String	String	String	Int32	String
1	A0A023VU42	Viral interleukin-10 homolog	unreviewed	185	{"taxI
2	A0A059WM99	Interleukin family protein	unreviewed	178	{"taxI
3	A0A061HZR3	Interleukin family protein	unreviewed	139	{"taxI
4	A0A068EG07	Viral interleukin-10 homolog	unreviewed	178	{"taxI
5	A0A068EGM0	Viral interleukin-10 homolog	unreviewed	174	{"taxI

Skills

SKILL.md files are included in ./skills for loading BioInfoAggregator.jl from local source and for fetching UniProt data by accession ID.

AI Usage Disclosure

Google Gemini was used for generating README.md and generating flow chart visual.

Cokelaer, T., Poinçot, T., Kouri, F., Pradhana, A., Cokelaer, M., & Saez-Rodriguez, J. (2013). BioServices: a Python package to access Biological Web Services. *Bioinformatics*, 29(24), 3241–3242. <https://doi.org/10.1093/bioinformatics/btt547>

Contributors, B. (2023). *BioFetch.jl: A Julia package for fetching biological data from web databases*. <https://github.com/BioJulia/BioFetch.jl>.

Greener, J. G., & Sternberg, M. J. E. (2021). BioStructures.jl: read, write and manipulate macromolecular structures in Julia. *Bioinformatics*, 37(11), 1618–1620. <https://doi.org/10.1093/bioinformatics/btaa1003>

Martin, F. J., Amode, M. R., Aneja, A., Austine-Orimoloye, O., Azov, A. G., Barnes, I., Becker, A., Bennett, R., Berry, A., Bhai, J., & others. (2024). Ensembl 2024. *Nucleic Acids Research*, 52(D1), D851–D858. <https://doi.org/10.1093/nar/gkad1049>

- ⁹⁵ Raasveldt, M., & M"uhleisen, H. (2019). DuckDB: an Embeddable Analytical Database.
⁹⁶ *Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19)*,
⁹⁷ 1981–1984. <https://doi.org/10.1145/3299869.3320212>
- ⁹⁸ The UniProt Consortium. (2023). UniProt: the Universal Protein Knowledgebase in 2023.
⁹⁹ *Nucleic Acids Research*, 51(D1), D523–D531. <https://doi.org/10.1093/nar/gkac1052>

DRAFT